Code style and lint

English version of ../../development/code-style.md.

What is checked and with what

Tool	Version	Configuration	Where it runs
ruff	ruff==0.17.3 (requirements-dev.txt)	ruff.toml	make lint, CI job "Tests, lint and a runnable app"
pytest	pytest==8.4.2	pytest.ini, tests/	make test, CI
gitleaks	8.28.0 (pinned in ci.yml)	.gitleaks.toml	CI job "No committed secrets"

Tool versions are pinned in requirements-dev.txt rather than installed as "the latest from PyPI": otherwise the same branch passes locally and fails in CI (or the other way round) because a rule set changed. The rule set is defined in ruff.toml instead of the default one, which makes the result identical in the editor, on the developer machine and on the runner.

Blocking rule set

select = ["E9", "F"] — what breaks execution and almost always points at a bug: syntax (E9), undefined names (F82), unused imports and variables (F401, F841). All of it is fixed in the code, so the check passes with zero findings.

Style debt (deliberately not blocking)

Set	Findings	Why it does not block
E4 (multiple imports per line)	~50	mechanical change, no behavioural effect
E7 (multiple statements per line)	~200	same legacy style, risky without tests per handler
BLE (broad except)	30+	deliberate in Flask handlers; narrow it one place at a time
SIM	~20	readability simplifications, not a priority

Tightening order: one rule → # noqa: <code> with a comment where the behaviour is intentional → a separate PR to narrow exceptions → add the rule to select. A repository wide --fix is not done in one go: it ruins git blame and incident analysis.

Agreements not checked automatically

- New code comes with tests; tests must not require the network.
- Background tasks (socket events) never write to the database bypassing app/user_db.py.
- Secrets are read from the environment only, defaults are empty.